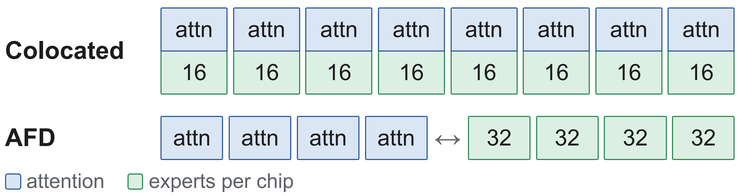
Attention–FFN disaggregation on TPU v6e

Charlie Xue, Akhil Chilaka, Rohan Anne, Jerry Ji

Motivation

- In Qwen3-30B-A3B, attention reads each user's KV cache, so its cost grows with the number of users, while the 128 experts are 58 GB of weights that every user shares.
- Colocated serving runs both on every chip, so they can't be scaled or batched separately.
- AFD puts them on different chips, as MegaScale-Infer and Step-3 already do on GPUs.



The obstacle on TPU

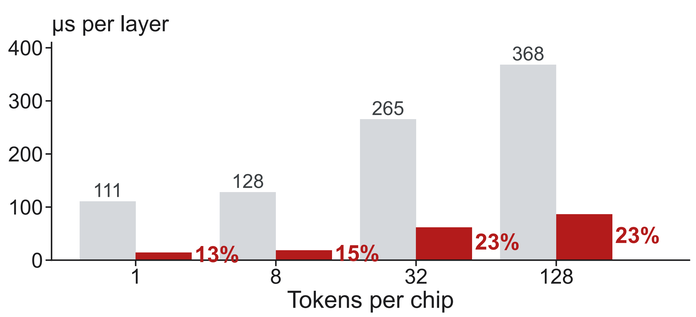
torch_tpu compiles one program and runs it on every chip, so attention chips and expert chips can't run different code. Our first attempts failed:

- Giving each group its own program made their shared communication step hang.
- torch_tpu doesn't support a runtime branch (`torch.cond`).
- An all-to-all that sends different amounts to each chip doesn't compile.
- Sending tokens chip to chip outside the compiled step works, but is about 90 times slower.

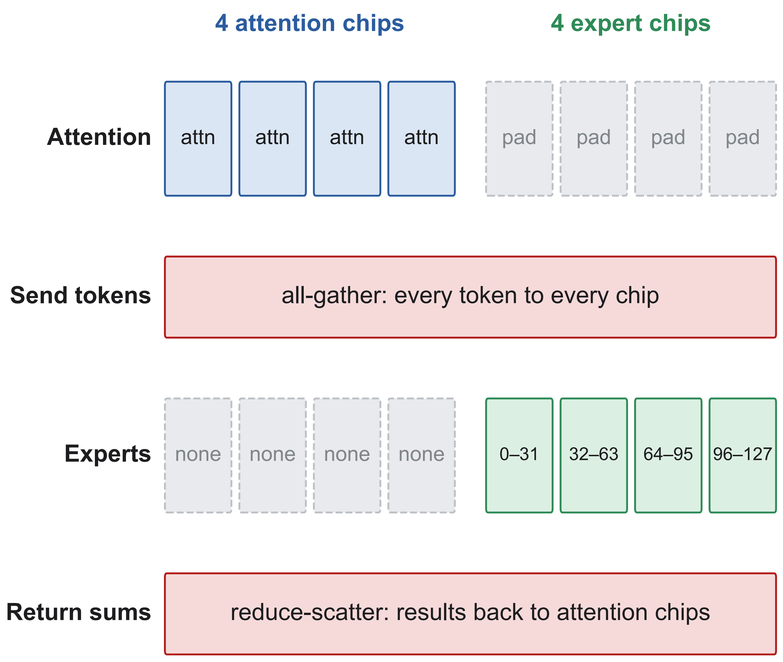
We ended up running the same program on every chip and passing each chip's role in as data.

Moving tokens is cheap

Timed on its own, sending tokens to the expert chips and back (**red**) takes 13 to 23% of the time of a colocated layer (grey).



Our design



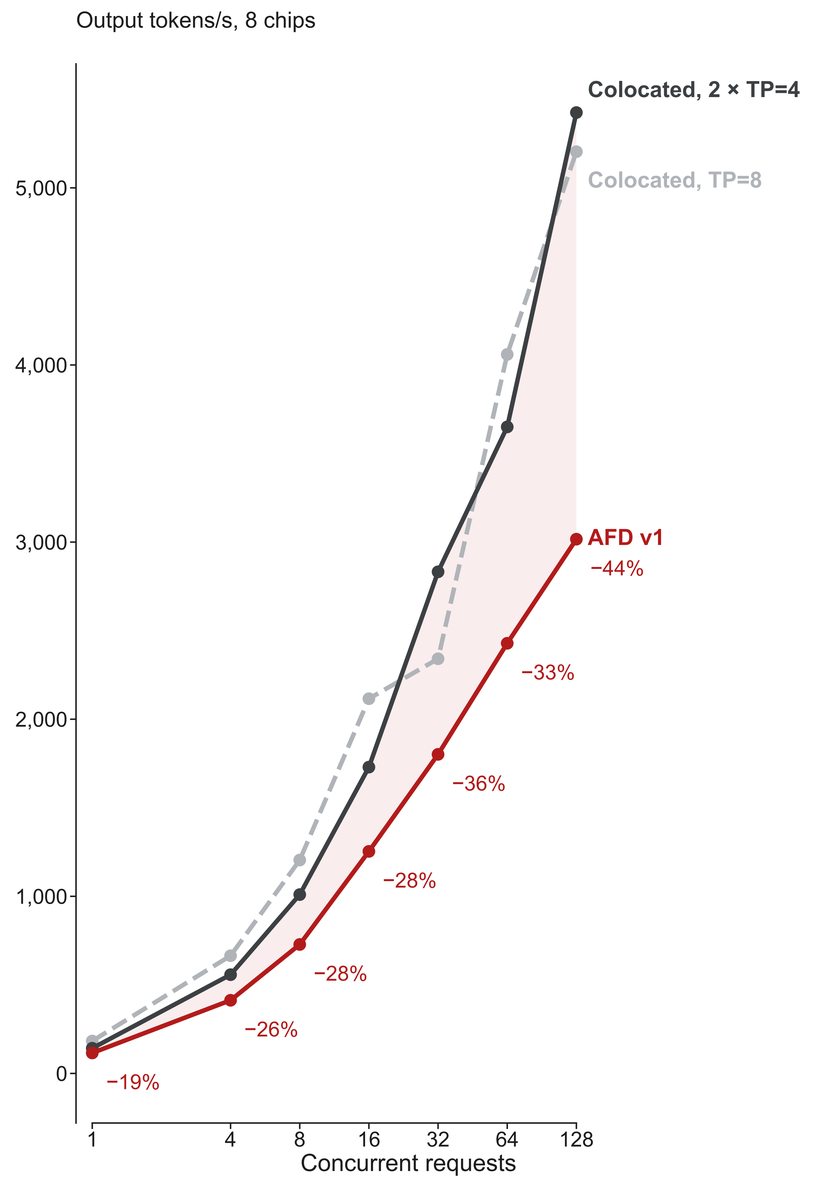
- With data parallelism, vLLM runs two copies of the model on 4 chips each, and both copies execute the same compiled program in lockstep.
- We send every request to the first copy and give its chips an empty expert range, which turns them into attention chips that skip the expert work.
- The second copy gets no requests, so its chips act as expert chips, each holding 32 of the 128 experts.
- In every layer, tokens travel to the expert chips and back through vLLM's existing all-gather and reduce-scatter, so we didn't need new communication code.
- The grey cells show wasted work, since the expert chips run attention on padding batches and each group waits while the other works.

AFD gives correct output

AFD gets all 4 of our known-answer prompts right, and where its text differs from colocated serving, the two candidate tokens had similar probabilities:

Prompt	AFD's token	Colocated's token
Apollo 11	**marked** .239	**achieved** .239
fibonacci(n)	**1** .525	**0** .463

AFD throughput is 19 to 44% below 2 × TP=4



Since moving tokens is cheap, most of this gap probably comes from the idle time in the grey cells, which we plan to confirm by profiling a single layer.

Next steps

- Profile a layer to measure how long each group sits idle
- Stop the expert chips from running attention on padding
- Split each batch in two so both groups can work at the same time
- Measure GSM8K accuracy and try other ways to divide the 8 chips